DocReverse

Reverse-engineering company documents into a model of how the organisation works

Project Report

Shayan Ponnanna Malchira

1. Summary

Companies explain how they work in many separate documents: policies, standard operating procedures, runbooks, spreadsheets, slide decks and e-mails. No single person holds the whole picture. Questions such as who owns a process, which systems it touches, what breaks if a system fails, or whether two documents disagree normally take hours of reading to answer.

DocReverse is a command-line project that reads those documents and rebuilds the operating model they describe. It splits each file into sections, finds the people, teams, systems and forms mentioned, works out who does what to what, recovers step-by-step processes, and stores everything in a local database and a knowledge graph. On top of that model it answers questions, traces the impact of a change, produces a full Markdown report with diagrams, and flags problems such as missing owners, contradictory numbers, broken references and stale documents.

The core runs entirely on the user's machine and needs nothing beyond Python. An optional layer uses an AI model for written answers and deeper extraction, with sensitive text redacted first.

> **Deliverables.** A Python package (`docreverse/`), a command-line tool, a test suite of 32 tests, a sample company document set with deliberately planted problems, an example generated report, and this report. Every extracted fact keeps the sentence it came from, so findings can be checked.

2. Problem and objectives

The problem

Knowledge about how a business operates is spread across documents written by different people at different times. The consequences are familiar: processes with no named owner, a policy that says three days while the procedure says five, references to documents that were retired, systems everyone uses but nobody has documented, and slow onboarding because the real workflow must be pieced together by hand.

Objectives

- Read the document formats a company actually has, without cloud services or special software.
- Recover structure: sections, processes, steps, owners, actors, systems, rules, numbers, definitions and cross-references.
- Represent the result as a graph so relationships can be queried and followed, not just searched.
- Detect inconsistencies and gaps automatically and show the evidence for each.
- Stay transparent and local by default, and make AI assistance optional and safe.

Scope

Supported inputs are `.txt`, `.md`, `.rst`, `.log`, `.html`, `.csv`, `.docx`, `.xlsx`, `.pptx`, `.eml` and `.pdf`. English-language business documents are the target. The system describes what documents say; it does not observe what people actually do.

3. How it works

Every file passes through the same six stages. Whatever the format, it first becomes a neutral list of blocks (headings, paragraphs, list items, table rows, formulas and comments), so later stages never depend on the file type.

Stage	What happens
1. Read	Format loaders turn each file into blocks.
2. Structure	Blocks become sections; the title, document code, type and header fields are found.
3. Extract	Each sentence is scanned for entities, relations, rules, numbers, references and terms.
4. Recover processes	Ordered steps become processes with actors, systems and an owner.
5. Store	SQLite holds all findings plus a full-text search index.
6. Analyse	A knowledge graph feeds questions, impact analysis, gap detection and reports.

A sentence, followed through

This is real output from the sample documents. Entity detection finds the role, the form and the system; the text between them is classified by a verb lexicon; and the person, not the form, is credited with using the system.

```
Employee creates the Expense Report in Concur and attaches all receipts.

entities   Employee (role), Expense Report (artifact), Concur (system)
relations  Employee creates Expense Report
           Employee uses Concur

The Expense Report is approved by the Line Manager.

relations  Line Manager approves Expense Report   (passive voice reversed)
```

Stage 1: reading files

- **Text and Markdown:** headings of all common styles, bullets versus numbered steps, wrapped list items, tables, code fences, and header lines such as `Owner: Finance Manager`.
- **HTML:** headings, paragraphs, ordered and unordered lists and tables; scripts and styles are dropped.
- **Word, PowerPoint, Excel:** parsed directly from their zipped XML with the standard library. Word keeps heading styles, real list numbering, tables and reviewer comments. PowerPoint keeps slide titles, bullets, tables and speaker notes. Excel keeps sheets as tables and every formula with a readable label, such as `Pricing!D4 (Total / Revenue) = SUM(D2:D3)`, because spreadsheets often hold the only written record of business logic.
- **CSV, e-mail, PDF:** delimiter detection; headers and body with quoted replies removed; PDF text with running headers and footers stripped. PDF needs `pypdf` or `pdftotext`, and scanned PDFs need OCR first.

Stage 2: structure

Blocks are grouped under headings into sections. The title, a document code such as `FIN-102`, the document type (policy, SOP, runbook, contract and so on) and header fields (owner, version, last reviewed) are detected.

Stage 3: extraction

Extraction is rule-based, so every result can be inspected and explained.

- **Entities** come in four kinds: role (Finance Manager), team (IT Service Desk), system (Concur) and artifact (Expense Report). Detection uses capitalised phrases that end in a head noun, lowercase phrases after a determiner ("the line manager"), tool names after cue words ("log into Concur"), and an optional company vocabulary file.
- **Relations** are found by classifying the text between two neighbouring entities. The vocabulary covers `approves`, `reviews`, `responsible_for`, `reports_to`, `sends_to`, `submits`, `creates`, `triggers`, `depends_on`, `uses` and `stored_in`. Passive voice reverses direction; a conjunction keeps the same subject ("logs into Concur and approves the report"); free-text objects are kept ("approves expenses over $5,000").
- **Rules** are sentences with must, shall, must not, should or may, sorted into obligations, prohibitions, recommendations and permissions.
- **Numbers** with units (money, durations such as "five business days", percentages), **cross-references** (`HR-204`, "see the Expense Policy") and **glossary terms** (acronym definitions, "X means Y").

Stage 4: processes

A section becomes a process when it contains at least two ordered steps under a process-like heading, at least three ordered steps anywhere, or enough sequence wording ("First… Then… Finally"). Each step records its actor, the systems it uses, and whether it is a decision. The owner comes from an `Owner:` field, an "owned by" sentence or a responsibility relation; if none exists, the most frequent actor is offered and clearly marked as inferred.

```
Expense Reimbursement SOP (recovered, shortened)
1  Employee creates the Expense Report in Concur
2  Employee submits the Expense Report to the Line Manager
3  Line Manager reviews the claim and approves it within 5 business days
4  If the claim is over $2,500, Finance Manager must also approve   [decision]
5  Accounts Payable verifies receipts and codes the expense in SAP
6  Payroll Team schedules payment
7  Employee receives a confirmation e-mail from Concur
```

Stages 5 and 6: storage and the graph

All findings go into one SQLite file. Re-running is incremental: a content hash per file decides whether it needs to be read again. The stored findings are then loaded into a knowledge graph whose nodes are roles, teams, systems, artifacts, processes and documents, and whose edges are the relations with their evidence sentences.

4. System design

Modules

Module	Responsibility
`ingest/`	One loader per format plus a registry that chooses by file extension
`structure.py`	Blocks to sections; title, code, type and header fields
`extract/entities.py`	Roles, teams, systems and artifacts; optional vocabulary file and aliases
`extract/relations.py`	Subject-verb-object triples with passive-voice handling
`extract/processes.py`	Steps, actors, owner, trigger and decisions
`extract/rules.py`, `facts.py`	Obligations and prohibitions; numbers, references and glossary
`pipeline.py`	Runs the analysis, chunks text for search, discovers files, ingests incrementally
`store.py`	SQLite schema and full-text search (FTS5 with BM25, with a fallback)
`model.py`, `graph.py`	Loads the store and builds the knowledge graph; impact, paths and exports
`analyze.py`	Overview, ownership, systems map, gap detectors, conflicts and explanations
`qa.py`	Retrieval plus graph facts, optionally a written answer
`report.py`	Full Markdown report with Mermaid diagrams
`llm.py`, `redact.py`	Optional AI calls; removal of sensitive tokens first
`cli.py`	All user commands

Impact analysis

Edges are interpreted by direction of effect. Flow edges (`sends_to`, `approves`, `triggers`, `creates`, `submits`, `responsible_for`) carry impact from source to target: if the source fails, the target is affected. Dependency edges (`uses`, `depends_on`, `references`, `reports_to`, `stored_in`) carry impact the other way: if the target fails, the source is affected. People and teams are shown as affected but are not chained through, so a system outage does not appear to affect unrelated work that one employee happens to do.

Design choices

- **Standard library only,** so it installs anywhere and is easy to audit. PDF and AI features are optional extras.
- **Rules before AI:** deterministic results, no cost, no data leaving the machine, and every finding explainable.
- **Evidence everywhere:** relations, conflicts and gaps point back to the original sentence and document.
- **One bad file never stops a batch:** failures are reported and skipped.

5. What it can tell you

Question	Command	Answered from
What processes exist, and what are their steps?	`processes`	Recovered step lists
How does a role, system or form work?	`explain`	Graph edges, processes, rules and definitions
What is affected if this changes or fails?	`impact`	Graph traversal in both directions
Where are the problems?	`gaps`	The detectors listed below
A free-form question	`ask`	Search results and graph facts, optionally AI
Everything in one document	`report`	All of the above, with diagrams

Problem detectors

Finding	Meaning
`ownerless_process`	A process with no stated owner
`inferred_owner`	The owner is only a guess from the steps; confirm it
`thin_process`	Fewer than three steps; detail may be missing
`broken_reference`	A document code (such as HR-204) that is not in the document set
`missing_document`	A named policy or procedure that was not found
`undocumented_system`	A system used in two or more documents but described in none
`undefined_acronym`	An acronym used three or more times and never defined
`stale_document`	Last reviewed or dated longer ago than the threshold (default 365 days)
`duplicate_process`	The same process in several documents, flagged more strongly if the owners differ
`conflicting_numbers`	Same unit, different value, in similar sentences (for example 3 versus 5 business days)

6. How to use it

Requirements and setup

Python 3.9 or newer. Unzip the project and work from its folder. Nothing needs installing for the core. Optional extras:

```
pip install pypdf               # read PDFs (or install poppler's pdftotext)
pip install anthropic           # AI answers and deep extraction
export ANTHROPIC_API_KEY=...    # only for the AI features
```

Quick start with the sample documents

```
python scripts/make_samples.py              # optional: creates docx, xlsx, pptx
samples
python -m docreverse ingest sample_docs     # reads everything,
builds .docreverse/index.db
python -m docreverse status
python -m docreverse processes
python -m docreverse processes --show "Expense" --mermaid
python -m docreverse explain "Line Manager"
python -m docreverse impact Concur
python -m docreverse impact SAP --direction up
python -m docreverse gaps
python -m docreverse ask "who approves expense claims over 2500"
python -m docreverse report --out out/report.md
python -m docreverse graph --format mermaid --out out/graph.mmd
python -m unittest discover -s tests -t .   # run the tests
```

Using it on your own documents

1. Run `python -m docreverse ingest /path/to/your/documents`. Subfolders are included. Run it again whenever files change; unchanged files are skipped. Add `--prune` to forget deleted files and `--force` to re-analyse everything.
2. Run `status` and `gaps` first. They show quickly how well the extraction fits your writing style.
3. If systems or roles are missed, create a vocabulary file and pass it with `--gazetteer company.json` (example below).
4. Use `explain`, `impact` and `ask` to investigate, and `report` to share an overview.
5. Treat every finding as a question to verify using the evidence sentence it shows.

```
{
  "systems": ["Workday", "PagerDuty"],
  "roles":   ["Incident Commander"],
  "teams":   ["Security Team"],
  "aliases": {"SAP ERP": "SAP", "Service Now": "ServiceNow"}
}
```

Command reference

Command	What it does	Useful options
`ingest PATH...`	Read files or folders and build the index	`--gazetteer FILE` `--force` `--prune` `--deep`
`status`	Counts of documents, processes, relations, rules and graph size	
`processes`	List processes, or show one in full	`--show NAME` `--mermaid`
`explain TOPIC`	Profile of a role, team, system, form or process	
`impact TOPIC`	What is affected, or what it depends on	`--direction down|up` `--depth N`
`gaps`	List all detected problems with evidence	`--stale-days N`
`ask QUESTION`	Answer a question from the documents	`--no-llm`
`search QUERY`	Full-text search with ranking	`-k N`
`graph`	Export the knowledge graph	`--format mermaid|dot|json` `--max-nodes N` `--out FILE`
`report`	Write the full Markdown report	`--out FILE`

The global option `--db PATH` (or the environment variable `DOCREVERSE_DB`) selects the index file, so separate projects can keep separate indexes.

Optional AI mode

With `ANTHROPIC_API_KEY` set and the `anthropic` library installed, `ask` writes a cited answer from the retrieved passages and graph facts, and `ingest --deep` asks the model to extract processes, relations, rules and terms section by section, marking them as coming from the model. The model name defaults to `claude-sonnet-5-5` and can be changed with `DOCREVERSE_MODEL`. Without a key, everything else works unchanged.

Outputs

- **report.md:** overview, document inventory, every process with a flowchart, a responsibility table, systems map, most critical dependencies, rules digest, glossary, all findings, and a graph diagram. Mermaid diagrams display in most Markdown viewers.
- **Graph exports:** Mermaid for documents, Graphviz DOT for diagrams, JSON for other tools.

7. Testing and results

Automated tests

32 tests run with the standard `unittest` module and all pass. They cover entity detection (including lowercase phrases and the company vocabulary), relation extraction (active, passive, conjunction, "submits X to Y", systems), numbers, rules, sentence splitting, every file loader (including Word, Excel and PowerPoint files built by the tests themselves), and an end-to-end run over the sample documents including search, impact analysis, the report and the redaction step.

Sample document set

The sample company has nine documents: an expense policy and SOP, an incident-response runbook, an onboarding procedure, a vendor-onboarding web page, a vendor scorecard (CSV), an access-request procedure (Word), a pricing model (Excel) and an operations review (PowerPoint). Problems were planted on purpose to check whether they are found.

Measure	Result
Documents / sections	9 / 27
Processes recovered	6
Relations / rules / glossary terms	33 / 11 / 2
Knowledge graph	72 nodes (24 roles, 13 teams, 16 artifacts, 8 systems, 6 processes, 5 documents) and 79 edges

Planted problem	Detected?
Policy says approval within 3 business days; SOP says 5	Yes, reported as `conflicting_numbers` with both sentences
SOP refers to HR-204, which is not in the document set	Yes, `broken_reference`
Documents last reviewed long ago	Yes, `stale_document`
The 7-step expense SOP, its owner and its decision step	Yes: owner Accounts Payable Supervisor, step 4 flagged as a decision, Concur and SAP found
A process inside a Word file; formulas in an Excel file	Yes: the access-request process is recovered; formulas are searchable with row and column labels

> **Read this before trusting the numbers.** These results come from a small synthetic document set written alongside the tool, so they show that the mechanisms work, not how accurate the tool is on real company documents. No precision or recall has been measured on real data. Before relying on it, run it on a few real documents and compare its findings with what you know.

8. Limitations

- **Heuristics, not understanding.** Expect missed relations and some wrong ones. The evidence sentence is always shown so each can be checked.
- **Naming.** Lowercase roles and systems are recognised only after a determiner or through the vocabulary file. A place name after "in" can be mistaken for a system.
- **Prose processes.** Procedures written as free text without steps or sequence words are missed; the optional AI mode helps.
- **Conflict detection** compares the similarity of sentences, so it can raise false alarms and can miss conflicts that are worded very differently.
- **Spreadsheets.** Formulas are indexed and searchable but not yet modelled as a dependency graph.
- **Scanned PDFs** have no text layer and need OCR first. Other languages are not supported.
- **Documents are not reality.** The model shows what is written, not what staff do. Use findings to ask better questions, not as verdicts.

9. Privacy and safety

- By default nothing leaves the machine. The index is a local file; keep it out of version control (the project already ignores `.docreverse/`).
- AI features need an explicit key and library. Before any text is sent, e-mail addresses, phone numbers, US social security numbers, card numbers that pass the Luhn check, IBANs and key-like secrets are replaced with placeholders. This is best effort, not a guarantee.
- Only analyse documents you are permitted to process, and share generated reports with the same care as the source documents, since they quote them.

10. Extending the project

- **New file format:** write a function that returns blocks and metadata, register it in `ingest/__init__.py` and add the extension to `config.py`.
- **New relation verb:** add a pattern to the lexicon in `extract/relations.py` and the predicate to the flow or dependency set in `graph.py`.
- **New check:** add a block to `Analyzer.gaps()` that returns findings.
- **Another language:** replace the role, verb and cue lists; the structure of the pipeline stays the same.

11. Future work

- OCR for scanned documents.
- A dependency graph of spreadsheet formulas.
- Semantic search with embeddings alongside keyword search.
- Comparing document versions to show how a process changed.
- Export of a responsibility matrix (responsible, accountable, consulted, informed).
- A small web interface over the report and graph.
- A labelled evaluation set from real documents to measure accuracy.

12. Conclusion

DocReverse turns a folder of documents into something that can be questioned: a set of recovered processes, a graph of who and what is connected to whom, and a list of places where the paperwork contradicts itself or is silent. It is local, transparent and extensible, and it shows its evidence.

Its limits are those of rule-based reading of text, so its output is best used as a map to verify, and as a fast way to find the questions worth asking.

Appendices

A. Project layout

```
docreverse_project/
  README.md  pyproject.toml  requirements.txt  sample_gazetteer.json
  docreverse/
    cli.py  config.py  models.py  textutil.py  structure.py
    pipeline.py  store.py  model.py  graph.py  analyze.py  qa.py  report.py
    llm.py  redact.py
    ingest/   text.py  html.py  office.py  other.py
    extract/  entities.py  relations.py  processes.py  rules.py  facts.py
  tests/        test_extraction.py  test_loaders.py  test_pipeline.py
                office_fixtures.py
  sample_docs/  nine sample documents
  scripts/      make_samples.py
  docs/         PROJECT_GUIDE.md
  out/          example_report.md  graph.mmd
```

B. Relation vocabulary

Predicate	Typical wording	Impact
`approves`	approves, signs off, is approved by	flow
`reviews`	reviews, audits, verifies, checks	flow
`responsible_for`	owns, manages, is accountable for	flow
`sends_to`	sends, escalates, notifies, forwards to	flow
`submits` / `creates`	submits, opens, raises, files, generates	flow
`triggers`	triggers, initiates, leads to	flow
`uses`	uses, logs into, records in, enters in	dependency
`depends_on`	depends on, requires, integrates with	dependency
`reports_to`	reports to	dependency
`stored_in`	is stored, recorded or saved in	dependency

C. Database tables

Table	Holds
`documents`, `sections`	One row per file and per section (title, code, type, header fields)
`mentions`	Every role, team, system and artifact found, with its sentence
`relations`	Subject, predicate, object, evidence and source (rules or model)
`processes`	Name, owner and how it was found, trigger, steps, systems and roles
`rules`, `facts`	Obligations and prohibitions; numbers with units
`refs`, `terms`	Cross-references; glossary definitions
`chunks`, `chunks_fts`	Text pieces and their full-text index